

List Comprehension in Python

1–10: Basic Transformations

1. Square of numbers from 1 to 10

```
[x**2 for x in range(1, 11)]
```

2. Even numbers from 1 to 20

```
[x for x in range(1, 21) if x % 2 == 0]
```

3. Odd numbers squared from 1 to 10

```
[x**2 for x in range(1, 11) if x % 2 != 0]
```

4. Double of elements in a list

```
lst = [1, 2, 3, 4]  
[x * 2 for x in lst]
```

5. Extract numbers divisible by 5

```
[x for x in range(1, 51) if x % 5 == 0]
```

6. Convert strings to uppercase

```
names = ['john', 'doe', 'alice']  
[name.upper() for name in names]
```

7. Get lengths of words in a list

```
words = ['apple', 'banana', 'cherry']  
[len(word) for word in words]
```

8. First letter of each word

```
[word[0] for word in words]
```

9. Filter names starting with 'A'

```
[name for name in names if name.startswith('a')]
```

10. Generate a list of tuples (number, square)

```
[(x, x**2) for x in range(1, 6)]
```

11–20: Conditional Logic

1. Replace even numbers with 0

```
[0 if x % 2 == 0 else x for x in range(1, 11)]
```

2. Flatten a list of lists

```
lst = [[1, 2], [3, 4], [5, 6]]  
[num for sublist in lst for num in sublist]
```

3. Generate multiples of 3 from 1 to 30

```
[x for x in range(1, 31) if x % 3 == 0]
```

4. Identify palindromes in a list

```
words = ['madam', 'apple', 'radar']  
[word for word in words if word == word[::-1]]
```

5. Filter positive numbers from a mixed list

```
lst = [-1, 2, -3, 4]  
[x for x in lst if x > 0]
```

6. Numbers ending with 7 between 1 and 100

```
[x for x in range(1, 101) if x % 10 == 7]
```

7. Skip elements containing 'a'

```
fruits = ['apple', 'kiwi', 'banana']  
[fruit for fruit in fruits if 'a' not in fruit]
```

8. Capitalize names with 5 letters

```
names = ["anshul", "harsh", "mummy"]  
[name.capitalize() for name in names if len(name) == 5]
```

9. Generate ASCII values for 'abc'

```
[ord(char) for char in 'abc']
```

10. Find prime numbers from 1 to 20

```
[x for x in range(2, 21) if all(x % i != 0 for i in range(2, x))]
```

21–30: Working with Strings

1. Remove vowels from a word

```
word = 'hello'  
[char for char in word if char not in 'aeiou']
```

2. Create a list of reversed words

```
words = ['abc', 'xyz']  
[word[::-1] for word in words]
```

3. Generate all 2-letter combinations

```
[x + y for x in 'ab' for y in 'cd']
```

4. Words longer than 3 characters

```
[word for word in words if len(word) > 3]
```

5. Filter words containing digit(s)

```
words = ['abc1', 'def', 'gh2']  
[word for word in words if any(char.isdigit() for char i  
n word)]
```

6. Remove punctuation from a sentence

```
sentence = 'Hello, world!'  
[char for char in sentence if char.isalnum() or char.iss  
pace()]]
```

7. Extract words starting with vowel

```
[word for word in words if word[0].lower() in 'aeiou']
```

8. Find common elements from two lists

```
a = [1, 2, 3]  
b = [2, 3, 4]  
[x for x in a if x in b]
```

9. Reverse and filter palindromes

```
[word[::-1] for word in words if word == word[::-1]]
```

10. Capitalize words not starting with vowels

```
[word.capitalize() for word in words if word[0].lower()  
not in 'aeiou']
```

31–50: Advanced List Comprehensions

1. Generate a list of even-length words

```
[word for word in words if len(word) % 2 == 0]
```

2. Square numbers only if divisible by 4

```
[x**2 for x in range(1, 21) if x % 4 == 0]
```

3. Check if a list contains only numbers

```
lst = ['1', 'a', '3']  
[x for x in lst if x.isdigit()]
```

4. Replace negatives with their absolute values

```
lst = [-1, 2, -3]  
[abs(x) for x in lst]
```

5. List all substrings of a word

```
word = 'abc'  
[word[i:j] for i in range(len(word)) for j in range(i+1,  
len(word)+1)]
```

6. Create a list of perfect squares < 100

```
[x**2 for x in range(1, 11)]
```

7. Filter prime numbers from a list

```
nums = [2, 3, 4, 5, 6]  
[x for x in nums if all(x % i != 0 for i in range(2,  
x))]
```

8. Multiply elements by their index

```
lst = [1, 2, 3]
```

```
[x * i for i, x in enumerate(lst)]
```

9. Generate Fibonacci numbers up to 10

```
fib = [0, 1]
[fib.append(fib[-1] + fib[-2]) or fib[-1] for _ in range(8)]
```

10. Generate a list of prime factors of 100

```
[x for x in range(2, 101) if 100 % x == 0]
```

51–60: Mathematical and Logical Transformations

1. Generate cubes of numbers from 1 to 10

```
[x**3 for x in range(1, 11)]
```

2. List all factors of a number (e.g., 36)

```
[x for x in range(1, 37) if 36 % x == 0]
```

3. Generate powers of 2 up to 1024

```
[2**x for x in range(11)]
```

4. Find common multiples of 3 and 5 between 1 and 100

```
[x for x in range(1, 101) if x % 3 == 0 and x % 5 == 0]
```

5. Find leap years between 2000 and 2100

```
[year for year in range(2000, 2101) if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0]
```

6. Identify perfect numbers from 1 to 1000

(A perfect number equals the sum of its proper divisors)

```
[x for x in range(1, 1001) if x == sum(i for i in range(1, x) if x % i == 0)]
```

7. Extract square roots (if perfect square)

```
import math
[x for x in range(1, 101) if math.sqrt(x).is_integer()]
```

8. Generate a multiplication table for 5

```
[5 * i for i in range(1, 11)]
```

9. Sum of even digits in a number

```
num = 123456
sum([int(d) for d in str(num) if int(d) % 2 == 0])
```

10. Create a list of Pythagorean triplets (a, b, c) for $c \leq 20$

```
[(a, b, c) for a in range(1, 21) for b in range(a, 21) for c in range(b, 21) if a**2 + b**2 == c**2]
```

61–70: Working with Dates and Time

1. Generate all months in a year as numbers

```
[month for month in range(1, 13)]
```

2. Extract weekdays from a date list

```
from datetime import datetime
dates = ['2024-10-15', '2024-10-16']
[date for date in dates if datetime.strptime(date, '%Y-%m-%d').weekday() < 5]
```

3. Generate all dates in October 2024

```
[f'2024-10-{{day:02}}' for day in range(1, 32)]
```

4. Filter weekends from a list of dates

```
[date for date in dates if datetime.strptime(date, '%Y-%m-%d').weekday() >= 5]
```

5. List all odd-numbered years from 2000 to 2050

```
[year for year in range(2000, 2051) if year % 2 != 0]
```

6. Generate minutes in an hour

```
[minute for minute in range(60)]
```

7. Filter years divisible by 100 but not by 400

```
[year for year in range(1500, 2100) if year % 100 == 0 and year % 400 != 0]
```

8. Find the number of Sundays in a month (October 2024)

```
[f'2024-10-{{day:02}}' for day in range(1, 32) if datetime(2024, 10, day).weekday() == 6]
```

9. Create 24-hour time format as a list of strings

```
[f'{{hour:02}}:00' for hour in range(24)]
```

10. Filter dates from a list falling on Fridays

```
[date for date in dates if datetime.strptime(date, '%Y-%m-%d').weekday() == 4]
```

71–80: Nested and Complex Lists

1. Generate a matrix of size 3×3 with 0s

```
[[0 for _ in range(3)] for _ in range(3)]
```

2. Transpose a matrix

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
[[row[i] for row in matrix] for i in range(3)]
```

3. Flatten a 3D list into 1D

```
lst_3d = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]  
[num for sublist1 in lst_3d for sublist2 in sublist1 for  
num in sublist2]
```

4. Filter diagonals from a 3×3 matrix

```
[matrix[i][i] for i in range(3)]
```

5. Generate identity matrix of size 4×4

```
[[1 if i == j else 0 for j in range(4)] for i in range  
(4)]
```

6. Reverse each row in a matrix

```
[row[::-1] for row in matrix]
```

7. Sum of each row in a matrix

```
[sum(row) for row in matrix]
```

8. Generate all pairs from two lists

```
a = [1, 2]  
b = [3, 4]  
[(x, y) for x in a for y in b]
```

9. Filter symmetric pairs from a list of tuples

```
pairs = [(1, 2), (2, 1), (3, 3)]  
[pair for pair in pairs if pair[0] == pair[1]]
```

10. Generate a multiplication table for numbers 1 to 5

```
[[i * j for j in range(1, 6)] for i in range(1, 6)]
```

81–100: Special Use Cases

1. Find all lowercase letters in a string

```
s = 'Hello World'  
[char for char in s if char.islower()]
```

2. Convert nested tuples to a flat list

```
nested_tuples = ((1, 2), (3, 4))  
[x for tup in nested_tuples for x in tup]
```

3. Generate list of odd sums from two lists

```
[x + y for x in [1, 3] for y in [2, 4] if (x + y) % 2 !=  
0]
```

4. Remove duplicates from a list

```
lst = [1, 2, 2, 3]  
[x for i, x in enumerate(lst) if lst.index(x) == i]
```

5. Generate n-letter prefixes of a word

```
word = 'python'  
[word[:i] for i in range(1, len(word)+1)]
```

6. Generate a list of prime numbers below 50

```
[x for x in range(2, 50) if all(x % i != 0 for i in range(2, int(x**0.5) + 1))]
```

7. Find palindromic numbers below 200

```
[x for x in range(1, 201) if str(x) == str(x)[::-1]]
```

8. Extract words with only alphabets

```
words = ['abc', '123', 'def']  
[word for word in words if word.isalpha()]
```

9. Generate a set of unique lengths from words

```
words = ['apple', 'banana', 'kiwi']  
{len(word) for word in words}
```

10. Identify elements appearing in both lists

```
a, b = [1, 2, 3], [2, 3, 4]  
[x for x in a if x in b]
```

1–10: Tuple Comprehension

1. Convert a list to a tuple of squares

```
tuple(x**2 for x in range(5))
```

2. Extract only even-indexed elements into a tuple

```
lst = [10, 20, 30, 40, 50]  
tuple(lst[i] for i in range(len(lst)) if i % 2 == 0)
```

3. Generate tuples of numbers and their cubes from 1 to 5

```
tuple((x, x**3) for x in range(1, 6))
```

4. Filter elements greater than 2 from a tuple

```
tup = (1, 2, 3, 4)
tuple(x for x in tup if x > 2)
```

5. Convert a list of tuples to a single flattened tuple

```
lst = [(1, 2), (3, 4)]
tuple(x for tup in lst for x in tup)
```

6. Find the sum of all elements in a tuple

```
tup = (1, 2, 3, 4)
sum(x for x in tup)
```

7. Generate a tuple of prime numbers below 20

```
tuple(x for x in range(2, 20) if all(x % i != 0 for i in
range(2, int(x**0.5) + 1)))
```

8. Create a tuple of reversed strings from a list

```
words = ['abc', 'def', 'ghi']
tuple(word[::-1] for word in words)
```

9. Filter out non-integer elements from a tuple

```
tup = (1, 'a', 2, 'b', 3.0)
tuple(x for x in tup if isinstance(x, int))
```

10. Generate a tuple of square roots (rounded) of numbers from 1 to 10

```
import math
tuple(round(math.sqrt(x), 2) for x in range(1, 11))
```

11–20: Set Comprehensions

1. Generate a set of even numbers from 1 to 20

```
{x for x in range(1, 21) if x % 2 == 0}
```

2. Extract unique words from a list of strings

```
words = ['apple', 'banana', 'apple', 'orange']  
{word for word in words}
```

3. Generate a set of squares from 1 to 10

```
{x**2 for x in range(1, 11)}
```

4. Find common elements in two lists using set comprehension

```
a = [1, 2, 3, 4]  
b = [3, 4, 5, 6]  
{x for x in a if x in b}
```

5. Generate a set of even-length strings from a list

```
words = ['hi', 'hello', 'python', 'code']  
{word for word in words if len(word) % 2 == 0}
```

6. Convert a list of tuples into a set of unique first elements

```
lst = [(1, 'a'), (2, 'b'), (1, 'c')]  
{x[0] for x in lst}
```

7. Find numbers divisible by both 2 and 3 up to 30

```
{x for x in range(1, 31) if x % 2 == 0 and x % 3 == 0}
```

8. Create a set of uppercase letters from a string

```
s = 'HeLLo WoRLd'  
{char for char in s if char.isupper()}
```

9. Filter non-alphanumeric characters from a string

```
s = 'Hello@123!'  
{char for char in s if not char.isalnum()}}
```

10. Generate a set of numbers where the sum of digits is even

```
{x for x in range(1, 50) if sum(int(d) for d in str(x))  
% 2 == 0}
```

21–35: Dictionary Comprehensions

1. Create a dictionary of squares from 1 to 10

```
{x: x**2 for x in range(1, 11)}
```

2. Invert a dictionary (swap keys and values)

```
d = {'a': 1, 'b': 2, 'c': 3}  
{v: k for k, v in d.items()}
```

3. Filter out dictionary items with values < 5

```
d = {'a': 1, 'b': 6, 'c': 3}  
{k: v for k, v in d.items() if v >= 5}
```

4. Generate a dictionary of characters and their ASCII values

```
{char: ord(char) for char in 'hello'}
```

5. Create a dictionary mapping numbers to their cubes

```
{x: x**3 for x in range(1, 6)}
```

6. Generate a dictionary with lengths of words as values

```
words = ['apple', 'banana', 'kiwi']  
{word: len(word) for word in words}
```

7. Convert a list of tuples into a dictionary

```
lst = [('a', 1), ('b', 2)]  
{k: v for k, v in lst}
```

8. Filter dictionary keys that contain vowels

```
d = {'apple': 5, 'ball': 3, 'cat': 2}  
{k: v for k, v in d.items() if any(c in 'aeiou' for c in k)}
```

9. Generate a frequency dictionary from a string

```
s = 'hello'  
{char: s.count(char) for char in set(s)}
```

10. Create a dictionary with square roots as values

```
import math  
{x: round(math.sqrt(x), 2) for x in range(1, 11)}
```

36–50: Advanced Nested Comprehensions

1. Flatten a list of lists using nested comprehension

```
lst = [[1, 2], [3, 4]]  
[num for sublist in lst for num in sublist]
```

2. Generate a dictionary from a list with indices as keys

```
lst = ['a', 'b', 'c']  
{i: val for i, val in enumerate(lst)}
```

3. Generate a multiplication table dictionary

```
{x: [x * i for i in range(1, 6)] for x in range(1, 6)}
```

4. Create a set of tuples with even and odd pairs

```
{(x, y) for x in range(5) for y in range(5) if (x + y) % 2 == 0}
```

5. Generate a dictionary of lists from two lists

```
keys = ['a', 'b', 'c']  
values = [1, 2, 3]  
{k: [v] for k, v in zip(keys, values)}
```

6. Generate tuples of matching indices from two lists

```
a = [1, 2, 3]  
b = [4, 2, 6]  
[(i, a[i]) for i in range(len(a)) if a[i] == b[i]]
```

7. Create a dictionary of factorials from 1 to 5

```
import math  
{x: math.factorial(x) for x in range(1, 6)}
```

8. Generate a set of palindromes from a list

```
words = ['madam', 'racecar', 'apple']  
{word for word in words if word == word[::-1]}
```